

Learning objectives

By the end of this unit, you should be able to:

- Explain the difference between centralized learning and federated learning
- Identify the roles of the **server** and the **clients** in Federated Learning
- Describe the Federated Learning training loop (communication rounds)
- Explain why Federated Learning is **not always preferable** to centralized learning
- Understand the role of Flower in a federated learning workflow

Assumptions and scope

We assume prior knowledge of:

- Supervised learning and model evaluation
- Basic Python-based machine learning workflows

We do **not** assume:

- Background in distributed systems
- Prior experience with Federated Learning

Out of scope for this unit:

- Differential Privacy
- Secure aggregation and cryptographic protocols
- Production-level deployment considerations

Introduction to Federated Learning

Federated Learning (FL) is a distributed machine learning paradigm in which multiple participants collaboratively train a shared global model without exchanging their raw data. Unlike traditional centralized learning approaches, the data remains local to each participant, and only model-related information is communicated during the training process.

Federated learning was first proposed by Google in 2016 and formally introduced in the work of McMahan et al. (2017). Since then, it has attracted significant attention and has been successfully applied in a wide range of domains, including healthcare (Hard et al., 2018), finance (Yoon et al., 2018), and natural language processing (Li et al., 2020), among others.

This paradigm has gained relevance due to the increasing importance of data privacy, regulatory constraints, and the natural distribution of data across organizations and devices. In many real-world scenarios—such as collaborative medical studies or large-scale mobile applications—centralizing data is either undesirable or impractical.

In a typical federated learning setup, a central server orchestrates the learning process by coordinating multiple clients. Each client performs local training using its own data and periodically sends model updates to the server, which aggregates them to produce an improved

global model. This global model is then redistributed to the clients, and the process is repeated over several communication rounds.

Despite its advantages, federated learning introduces challenges that do not appear in centralized settings. Data across clients is often heterogeneous and non-identically distributed, communication between clients and server can be costly, and not all clients may be available at every training round.

In this notebook, we will introduce the fundamental concepts of federated learning and simulate a simple federated scenario. The goal is to build an intuitive understanding of how federated learning differs from centralized training, laying the foundation for implementing and analyzing a complete federated learning algorithm in the next session.

Centralized Learning vs. Federated Learning

In traditional centralized machine learning, all training data is collected and stored in a single location, where the model is trained using the complete dataset. This approach has been the dominant paradigm for many years and underlies most standard machine learning pipelines, as it simplifies both model training and evaluation.

Federated learning departs from this paradigm by keeping the data decentralized. Instead of transferring data to a central server, the learning process is distributed across multiple clients, each of which trains a local model using its own data. The central server is responsible for coordinating the process and aggregating the information received from the clients in order to update a shared global model.

From a methodological perspective, centralized learning benefits from direct access to the full data distribution, which often leads to stable optimization and straightforward convergence behavior. In contrast, federated learning must cope with data that is fragmented across clients and often follows heterogeneous, non-identically distributed patterns. As a result, optimization becomes more challenging and model updates may reflect local biases present in individual clients.

Beyond methodological considerations, the two paradigms differ significantly in terms of data governance and system design. Centralized learning requires data transfer and storage in a single repository, which may raise privacy, legal, or ethical concerns. Federated learning, on the other hand, reduces the need for raw data sharing and is therefore better suited to scenarios where data cannot be centralized due to regulatory constraints or institutional boundaries.

Understanding these differences is essential for appreciating both the potential and the limitations of federated learning. In the following sections, we will simulate a federated setting to highlight how decentralization and data heterogeneity affect the learning process in

Key Components of a Federated Learning System

A federated learning system is composed of several fundamental components that interact throughout the training process. Understanding these components is essential before examining how the federated learning algorithm operates over time.

- **Client**
A client refers to a device or edge node that holds a local dataset and actively participates in the training of the federated model. Each client performs local computation using its private data and communicates model updates to the server.
- **Server**
The server is the central coordinating entity responsible for managing the federated learning process. It distributes the global model to selected clients, receives their model updates, and aggregates them to produce a new version of the global model.
- **Federated dataset**
The federated dataset consists of multiple decentralized local datasets, each owned by a different client. These datasets are not directly accessible by the server and are used collaboratively to train the federated model.
- **Federated model**
The federated model is the machine learning model that is collaboratively trained using the federated dataset. Although training is performed in a decentralized manner, the goal is to obtain a global model that generalizes well to new data while preserving the privacy of each client's local data.
- **Federated optimization**
Federated optimization refers to the process of training the federated model using decentralized data and model updates from the clients. This optimization setting differs from classical centralized optimization due to data heterogeneity, partial client participation, and communication constraints.
- **Aggregation**
Aggregation is the process by which the server combines the model updates received from multiple clients into a new global model. This step is crucial for incorporating information learned across clients and is commonly implemented using weighted averaging or related techniques.
- **Rounds**
Rounds refer to the iterative cycles of communication in federated learning. In each round, the global model is distributed to clients, locally updated, aggregated, and redistributed. Training proceeds over multiple rounds until a stopping criterion is reached.

With these components in mind, we can now describe the typical phases that govern the execution of a federated learning algorithm.

Phases of a Federated Learning Algorithm

Once the main components of a federated learning system have been defined, the training process can be described as a sequence of iterative phases that are repeated over multiple communication rounds. These phases determine how the federated model is collaboratively trained while keeping the data decentralized.

1. **Initialization of the global model**

The federated learning process starts with the initialization of a global model on the server. This model serves as the common starting point for all clients and may be initialized randomly or using pre-trained weights.

2. **Distribution of the global model to clients**

At the beginning of each round, the server selects a subset of available clients and sends them the current version of the global model. Client selection strategies may vary depending on system constraints, availability, or efficiency considerations.

3. **Local training on client data**

Each selected client performs local optimization of the received model using its own federated dataset. Training is typically carried out for a limited number of local epochs to balance learning progress and communication cost.

4. **Communication of local updates to the server**

After completing local training, clients transmit their model updates back to the server. These updates may consist of the full model parameters or parameter differences relative to the received global model.

5. **Aggregation of client updates**

The server aggregates the updates received from the clients to produce a new global model. This aggregation step integrates information learned across the decentralized datasets and is commonly implemented using weighted averaging based on the size of each client's dataset.

6. **Iteration over communication rounds**

The updated global model is redistributed to the clients, and the process is repeated over multiple rounds. Training continues until a stopping criterion is met, such as convergence of the global model or reaching a predefined number of rounds.

This iterative procedure defines the core workflow of most federated learning algorithms and will serve as the conceptual basis for the simulations and implementations explored in this notebook.

Introduction to Flower (Flwr)

In order to experiment with federated learning algorithms in practice, it is necessary to rely on software frameworks that abstract away much of the complexity associated with distributed systems, client coordination, and communication. One such framework is Flower (Flwr), a flexible and lightweight open-source framework specifically designed for federated learning.

It is important to emphasize that Flwr is not a machine learning algorithm. Instead, it is a software framework that provides the infrastructure required to implement, simulate, and deploy federated learning algorithms. The learning algorithm itself—such as the model architecture, loss function, and optimization strategy—remains under the control of the user.

Flwr follows a modular and framework-agnostic design, allowing developers and researchers to integrate federated learning workflows with popular machine learning libraries such as PyTorch, TensorFlow, or NumPy. Rather than enforcing a specific algorithmic formulation, Flwr enables users to define custom client behavior, server-side logic, and aggregation strategies.

From a conceptual standpoint, Flwr closely reflects the components and phases discussed in the previous sections. It explicitly models the interaction between a central server and multiple clients, supports iterative communication rounds, and facilitates the exchange and aggregation of model updates. This clear correspondence between theory and implementation makes Flwr particularly well suited for educational and experimental purposes.

In this notebook, Flwr will be used to simulate a federated learning environment on a single machine. This approach allows us to focus on the fundamental principles of federated learning without requiring a fully distributed infrastructure. In the following sections, we will introduce the basic elements of Flwr and use them to construct a simple federated learning scenario.

To use Flower for federated learning experiments, the library must first be installed and imported in the Python environment:

```
import flwr as fl
import tensorflow as tf

print("Flower:", fl.__version__)
print("TensorFlow:", tf.__version__)

Flower: 1.27.0
TensorFlow: 2.19.0

# Attempt to import Flower (Flwr). If it is not installed, install it in simulation mode.
try:
    import flwr as fl
except ImportError:
    !pip install flwr[simulation]
    !pip install ray[default] # Required dependency for Flower in simulation mode
    import flwr as fl # Import again after installation
```

```
# Attempt to import TensorFlow. Install it if not available.
try:
    import tensorflow as tf
except ImportError:
    !pip install tensorflow
    import tensorflow as tf # Import again after installation
```

When setting up a federated learning simulation environment with Flower, it is important to install the framework using the `simulation` extra, that is, `flwr[simulation]`. This option ensures that all additional dependencies required to simulate a federated learning environment on a single machine are correctly installed.

In particular, Flower relies on the `flwr_simulation` module to orchestrate multiple simulated clients within a single Python process. Additionally, starting from version 1.15, Flower uses `ray` as a backend for its simulation mode. For this reason, `ray` must also be available in the environment when running federated learning simulations.

It is worth noting that this setup is specific to simulation-based experiments. In a fully distributed deployment, where the server and clients run on separate machines, Flower should be installed using `pip install flwr` on both the server and all client devices to ensure compatibility and consistency across the system.

For the most up-to-date and comprehensive installation instructions, refer to the official Flower documentation: [Link to official Flower documentation](#)

After installing the `flwr` package, it can be imported into Python as shown below.

```
import flwr as fl
import tensorflow as tf
```

Flower provides a set of classes and abstractions that can be used to define a federated learning environment, coordinate training across clients, and evaluate a global model. Rather than prescribing a specific machine learning algorithm, Flower allows users to integrate their own models and training logic using standard machine learning libraries.

An important practical consideration in federated learning is that the model must be serializable, as model parameters need to be transmitted between the server and the clients during training. As a consequence, not all model types are equally suitable for federated learning, particularly in simulation or distributed settings.

In this notebook, we will use an Artificial Neural Network (ANN) implemented with TensorFlow and Keras. Keras models are lightweight, widely used, and well-suited for serialization, making them a convenient choice for illustrating federated learning concepts.

For more details on how Flower integrates with TensorFlow and Keras, refer to the official Flower documentation: <https://flower.dev/docs/quickstart-tensorflow.html>

We now define a simple neural network model that each federated client will use.

```

# Define a simple neural network model using TensorFlow and Keras
def generate_ann():
    model = tf.keras.Sequential(
        [
            tf.keras.layers.Input(shape=(32, 32, 3)), # Input: 32x32
            RGB images
            tf.keras.layers.Flatten(),
            tf.keras.layers.Dense(64, activation="relu"), # Hidden
            layer
            tf.keras.layers.Dense(64, activation="relu"), # Hidden
            layer
            tf.keras.layers.Dense(10, activation="softmax"), #
            Output: 10 classes
        ]
    )

    model.compile(
        loss=tf.keras.losses.sparse_categorical_crossentropy,
        optimizer=tf.keras.optimizers.Adam(),
        metrics=["accuracy"],
    )

    return model

```

At this point, the model definition is identical to what we would use in centralized learning. What do you expect to change when moving to a federated setting?

As we will train a deep learning model with TensorFlow, it is convenient to work with `tf.data.Dataset` objects, since they enable efficient input pipelines and can take advantage of hardware acceleration (e.g., GPUs) when available.

However, in Flower simulation mode it is often easier (and more robust) to keep the *data containers exchanged between components* as simple and serializable Python objects (e.g., NumPy arrays or tuples). For that reason, in this notebook we will first load and partition the data manually into per-client splits using NumPy arrays. Later, each client will convert its local split into `tf.data.Dataset` objects for training and evaluation.

```

import numpy as np
import tensorflow as tf

NUM_CLIENTS = 3
RANDOM_SEED = 42

def unison_shuffled_copies(a: np.ndarray, b: np.ndarray, seed: int =
RANDOM_SEED):
    """Shuffle two arrays in unison, preserving alignment between
    inputs and labels."""
    assert len(a) == len(b)
    rng = np.random.default_rng(seed)
    p = rng.permutation(len(a))

```

```

    return a[p], b[p]

def split_index(a: np.ndarray, n: int):
    """Return a list of index arrays splitting 'a' into 'n'
    approximately equal parts."""
    return np.array_split(np.arange(len(a)), n)

def load_datasets(num_clients: int, train_size: int = 10_000,
test_size: int = 1_000):
    """
    Load CIFAR-10, normalize, shuffle, and split it into per-client
    (train, val, test) tuples.

    Returns:
        train_splits: list of (x_train_i, y_train_i) for each client
        val_splits:    list of (x_val_i,    y_val_i)    for each client
        test_splits:  list of (x_test_i,   y_test_i)   for each client
    """
    (x_train, y_train), (x_test, y_test) =
tf.keras.datasets.cifar10.load_data()

    # Normalize to [0, 1]
    x_train = x_train.astype("float32") / 255.0
    x_test = x_test.astype("float32") / 255.0

    # Keep the notebook lightweight (faster for classroom execution)
    x_train, y_train = x_train[:train_size], y_train[:train_size]
    x_test, y_test = x_test[:test_size], y_test[:test_size]

    # Flatten labels from shape (N, 1) to (N,) to simplify downstream
    handling
    y_train = y_train.squeeze()
    y_test = y_test.squeeze()

    # Shuffle (fixed seed for reproducibility in class)
    x_train, y_train = unison_shuffled_copies(x_train, y_train,
seed=RANDOM_SEED)
    x_test, y_test = unison_shuffled_copies(x_test, y_test,
seed=RANDOM_SEED + 1)

    # Split indices per client
    train_index = split_index(x_train, num_clients)
    test_index = split_index(x_test, num_clients)

    train_splits, val_splits, test_splits = [], [], []

    for cid in range(num_clients):
        client_train_idx = train_index[cid]
        client_test_idx = test_index[cid]

```

```

    # Per-client train split
    x_c, y_c = x_train[client_train_idx],
    y_train[client_train_idx]

    # 10% validation (kept simple and deterministic)
    val_size = max(1, len(x_c) // 10)
    x_val, y_val = x_c[:val_size], y_c[:val_size]
    x_tr, y_tr = x_c[val_size:], y_c[val_size:]

    train_splits.append((x_tr, y_tr))
    val_splits.append((x_val, y_val))
    test_splits.append((x_test[client_test_idx],
    y_test[client_test_idx]))

    return train_splits, val_splits, test_splits

trainloaders, valloaders, testloaders = load_datasets(NUM_CLIENTS)

```

Before proceeding to a federated learning setup, it is good practice to validate the data pipeline and model definition in a simpler setting. As a sanity check, we first train and evaluate the model using the local dataset of a single client. It is like a centralized train to check the correctness.

This step serves two main purposes. First, it verifies that the model can be successfully trained using the client's data, ensuring that the model architecture, loss function, and optimizer are correctly defined. Second, it helps identify potential issues in the data loading and preprocessing steps early, before introducing the additional complexity of federated learning.

```

# Instantiate a fresh model
model = generate_ann()

# Select the data of a single client (client_id = 0)
x_train_c, y_train_c = trainloaders[0]
x_val_c, y_val_c = valloaders[0]

# Train the model locally (single-client, centralized setting)
model.fit(
    x_train_c,
    y_train_c,
    epochs=1,
    batch_size=32,
    verbose=1,
)

# Evaluate on the client's validation set
loss, accuracy = model.evaluate(x_val_c, y_val_c, verbose=0)

print(f"Validation loss: {loss:.4f}")
print(f"Validation accuracy: {accuracy * 100:.2f}%")

```

```
94/94 ————— 2s 6ms/step - accuracy: 0.1949 - loss: 2.1907
Validation loss: 2.0036
Validation accuracy: 26.43%
```

At this point, we move from local, single-client training to a federated learning setting. Conceptually, the local training procedure remains the same, but it is now executed independently by multiple clients and coordinated by a central server.

In this notebook, we use **Flower in simulation mode**, which allows us to simulate the interaction between a server and multiple clients on a single machine. This approach avoids the need to launch separate processes or machines, while preserving the logical structure of a federated learning system.

In Flower, the **server** is responsible for coordinating the training process. It selects clients for each communication round, distributes the current global model parameters, and aggregates the updates returned by the clients. Each **client** performs local training using its private dataset and reports the results back to the server.

The interaction between server and clients is defined through a client interface. When a client is selected during a training round, the server invokes specific methods on the client to carry out local training and evaluation. These method calls conceptually correspond to network communication in a fully distributed setup, but are simulated locally in this notebook.

For TensorFlow and Keras-based workloads, Flower provides a convenient base class called `NumPyClient`. This class simplifies the implementation of the client interface by allowing model parameters to be exchanged as NumPy arrays. A `NumPyClient` typically implements three methods:

Note: Flower supports both simulation-based experiments and fully distributed deployments. In this notebook, we exclusively work in **simulation mode**.

```
# Define a Flower client by extending NumPyClient
class MyClient(fl.client.NumPyClient):
    def __init__(self, model, train_data, val_data):
        self.model = model
        self.train_data = train_data  # tuple (x_train, y_train)
        self.val_data = val_data      # tuple (x_val, y_val)

    def get_parameters(self, config):
        # Return the current local model parameters
        return self.model.get_weights()

    def fit(self, parameters, config):
        # Update local model with global parameters
        self.model.set_weights(parameters)

        x_train, y_train = self.train_data
        self.model.fit(
            x_train,
```

```

        y_train,
        epochs=1,
        batch_size=32,
        verbose=0,
    )

    # Return updated parameters and number of training examples
    return self.model.get_weights(), len(x_train), {}

def evaluate(self, parameters, config):
    # Update local model with global parameters
    self.model.set_weights(parameters)

    x_val, y_val = self.val_data
    loss, accuracy = self.model.evaluate(x_val, y_val, verbose=0)

    # Return loss, number of evaluation examples, and metrics
    return float(loss), len(x_val), {"accuracy": float(accuracy)}

```

In Flower, clients can be created by extending either the `flwr.client.Client` or `flwr.client.NumPyClient` classes. In the previous example, we used `NumPyClient` because it is easier to implement and requires less code as a template. Along with the extended class, there are three main methods that need to be implemented:

- `get_parameters`: Returns the current local model parameters.
- `fit`: Receives model parameters from the server, trains the model parameters on the local data, and returns the (updated) model parameters to the server.
- `evaluate`: Receives model parameters from the server, evaluates the model parameters on the local data, and returns the evaluation result to the server.

As you can see, the `MyClient` class implemented in the previous example follows this same structure.

In simulation mode, Flower needs a way to instantiate a client given its identifier (`cid`). The function `client_fn` acts as a factory: for each simulated client, it selects the corresponding local data partition, creates a fresh model, and returns a `NumPyClient` instance.

A key detail is that each client should have its own model instance. Even though all clients share the same architecture, their parameters will evolve differently during local training before being aggregated on the server.

Sometimes, especially when we are simulating multiple clients on a single device, it can be useful to use a function to create the client when it is required. This is particularly important in stateless frameworks, such as PyTorch, which can benefit from a more efficient implementation that creates clients only when they are required for training or evaluation. For example, the following code loads different examples for each client before discarding them:

```

from flwr.common import Context

def client_fn(context: Context) -> fl.client.Client:

```

```

"""Create a Flower client for a given simulated node."""
partition_id = context.node_config.get("partition-id",
context.node_id)
cid = int(partition_id)

# Each client gets its own model instance
model = generate_ann()

# Select this client's local data partition
train_data = trainloaders[cid]
val_data = valloaders[cid]

# Return a Flower client
return MyClient(model, train_data, val_data).to_client()

```

Running Flower as an App (ClientApp + ServerApp)

Recent versions of Flower encourage organizing federated learning projects as a **Flower App**. In this setup, the client-side logic is wrapped in a `ClientApp` and the server-side orchestration is wrapped in a `ServerApp`. This structure is the recommended approach for both simulation and distributed deployments.

In simulation mode, we can execute the app directly from this notebook using `run_simulation(...)`, which runs a `ServerApp` together with multiple `ClientApp` instances (one per simulated node). To do so we would require the following code:

```

from flwr.clientapp import ClientApp

client_app = ClientApp(client_fn=client_fn)

```

Server-side Strategy and Simulation

One point to highlight is that the framework is not only going to manage the `losses_distributed`, but none of the other metrics. Due to the diverse treatment of those measures, the framework cannot accurately handle the aggregation of these metrics. Users need to tell the framework how to handle and aggregate these custom metrics.

The strategy will then call these functions whenever it receives fit or evaluates metrics from clients. The two possible functions are `fit_metrics_aggregation_fn` and `evaluate_metrics_aggregation_fn`. For example, the following code creates the weighted average, and the previous example can be adapted as follows:

```

from typing import List, Tuple, Dict
from flwr.common import Metrics

def weighted_average(metrics: List[Tuple[int, Metrics]]) -> Metrics:
    if not metrics:

```

```

        return {}

    accuracies = [num_examples * m["accuracy"] for num_examples, m in
metrics]
    examples = [num_examples for num_examples, _ in metrics]
    return {"accuracy": sum(accuracies) / sum(examples)}

```

In Flower simulation mode, we do not explicitly start a server process. Instead, we configure the *server-side strategy* and let Flower orchestrate the full federated workflow (server coordination, client selection, parameter distribution, and aggregation) within a single Python process.

The most common baseline strategy is **Federated Averaging (FedAvg)**. At each round, the server selects a fraction of the available clients, sends them the current global model parameters, receives locally-updated parameters after training, and aggregates them (typically via a weighted average) to produce the next global model.

In the next cell, we configure a FedAvg strategy and start a federated learning simulation with `NUM_CLIENTS` clients for a small number of rounds.

```

from flwr.server import ServerApp, ServerAppComponents

num_rounds = 3

def server_fn(context: Context):
    # Instantiate the model to obtain initial parameters
    model = generate_ann()
    initial_weights = model.get_weights()
    del model # free memory early

    # Convert to flwr Parameters
    initial_parameters =
fl.common.ndarrays_to_parameters(initial_weights)

    # Create FedAvg strategy
    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0,
        fraction_evaluate=0.5,
        min_fit_clients=NUM_CLIENTS,
        min_evaluate_clients=max(1, NUM_CLIENTS // 2),
        min_available_clients=NUM_CLIENTS,
        initial_parameters=initial_parameters,
        evaluate_metrics_aggregation_fn=weighted_average,
    )

    # Define ServerConfig
    config = fl.server.ServerConfig(num_rounds=num_rounds)

    return ServerAppComponents(strategy=strategy, config=config)

```

```
# Create Server
```

```
server_app = ServerApp(server_fn=server_fn)
```

Once everything is implemented, we can run the simulation

```
from flwr.simulation import run_simulation
```

```
history = run_simulation(  
    server_app=server_app,  
    client_app=client_app,  
    num_supernodes=NUM_CLIENTS, # number of simulated nodes/clients  
)
```

```
INFO :      Starting Flower ServerApp, config: num_rounds=3, no  
round_timeout  
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:  
DeprecationWarning: datetime.datetime.utcnow() is deprecated and  
scheduled for removal in a future version. Use timezone-aware objects  
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).  
    return datetime.datetime.utcnow().replace(tzinfo=utc)
```

```
INFO :  
INFO :      [INIT]  
INFO :      Using initial global parameters provided by strategy  
INFO :      Starting evaluation of initial global parameters  
INFO :      Evaluation returned no results (`None`)  
INFO :  
INFO :      [ROUND 1]  
INFO :      configure_fit: strategy sampled 3 clients (out of 3)  
(pid=gcs_server) [2026-03-15 14:16:56,118 E 30334 30334] (gcs_server)  
gcs_server.cc:302: Failed to establish connection to the event+metrics  
exporter agent. Events and metrics will not be exported. Exporter  
agent status: RpcError: Running out of retries to initialize the  
metrics agent. rpc_code: 14  
(pid=30670) 2026-03-15 14:16:58.654381: E  
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to  
register cuFFT factory: Attempting to register factory for plugin  
cuFFT when one has already been registered  
(pid=30670) WARNING: All log messages before absl::InitializeLog() is  
called are written to STDERR  
(pid=30670) E0000 00:00:1773584218.693648 30670 cuda_dnn.cc:8579]  
Unable to register cuDNN factory: Attempting to register factory for  
plugin cuDNN when one has already been registered  
(pid=30670) E0000 00:00:1773584218.705653 30670 cuda_blas.cc:1407]  
Unable to register cuBLAS factory: Attempting to register factory for  
plugin cuBLAS when one has already been registered  
(pid=30670) W0000 00:00:1773584218.750389 30670  
computation_placer.cc:177] computation placer already registered.  
Please check linkage and avoid linking the same target more than once.  
(pid=30670) W0000 00:00:1773584218.750442 30670
```

```
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=30670) W0000 00:00:1773584218.750445 30670
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=30670) W0000 00:00:1773584218.750447 30670
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(raylet) [2026-03-15 14:17:01,352 E 30422 30422] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
(ClientAppActor pid=30670) 2026-03-15 14:17:08.857779: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
INFO :      aggregate_fit: received 3 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(ClientAppActor pid=30670) [2026-03-15 14:17:26,510 E 30670 30722]
core_worker_process.cc:825: Failed to establish connection to the
metrics exporter agent. Metrics will not be exported. Exporter agent
status: RpcError: Running out of retries to initialize the metrics
agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 3 round(s) in 73.25s
INFO :      History (loss, distributed):
INFO :          round 1: 2.258411407470703
INFO :          round 2: 2.0568113327026367
INFO :          round 3: 1.966659426689148
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.14714714884757996),
INFO :                  (2, 0.18918919563293457),
INFO :                  (3, 0.2222222238779068)]}
INFO :
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
```

Note: *Be aware that the simulation is a really resource consuming task. You can get some errors and warnings linked to that. There are different [configurations](#) that you can apply to configure the available resources for simulating the process.*

Inspecting the First Federated Learning Results

After running the federated learning simulation, we can analyse the results across the communication rounds. At this stage, the goal is not to achieve high accuracy, but to understand how the federated learning process behaves and how it differs from centralised training.

The `history` object returned by Flower should contain information about the evolution of the global model during training and evaluation. By inspecting these results, we can gain insight into how the model improves across rounds and how aggregation of client updates affects performance.

Depending on your version of flower, the results can be `None` in the return of the simulation, but for utility reasons, we can define the following function:

```
def print_history(h):
    if h is None:
        print("history is None (this Flower version may not return a
History object from run_simulation).However, you can check the
run_simulation logs")
        return

    # Distributed loss
    if getattr(h, "losses_distributed", None):
        print("Distributed loss:")
        for r, loss in h.losses_distributed:
            print(f" Round {r}: loss = {loss:.4f}")

    # Distributed metrics
    if getattr(h, "metrics_distributed", None):
        print("\nDistributed metrics:")
        for metric_name, series in h.metrics_distributed.items():
            print(f" {metric_name}:")
            for r, value in series:
                if isinstance(value, float):
                    print(f" Round {r}: {value * 100:.2f}%")
                else:
                    print(f" Round {r}: {value}")

print_history(history)
```

```
history is None (this Flower version may not return a History object from run_simulation). However, you can check the run_simulation logs
```

Discussion

Several observations can be made from these initial results:

- The performance of the global model typically improves over communication rounds, although convergence may be slower than in centralized training.
- The accuracy obtained after a small number of rounds is often lower than what we observed during the single-client sanity check.
- Variability across clients and limited local training can lead to noisy or unstable updates, especially in early rounds.

These behaviors are expected and highlight some of the fundamental challenges of federated learning, such as data heterogeneity and limited communication. At this point, it is useful to reflect on how design choices—such as the number of rounds, the fraction of participating clients, or the amount of local training—may influence the final performance.

Questions for reflection:

- How do these results compare to the single-client experiment performed earlier?
- Why might federated learning require more training rounds to reach similar performance?
- What factors could explain differences in convergence speed or stability?

Answers

- Our federated learning results were generally weaker than the single-client experiment. In the single-client setting, training was faster and more stable because the model learned from one complete dataset directly. In federated learning, the data was split across clients, so the server only combined local updates, which made learning slower and less stable.
- Federated learning usually needs more rounds because the model does not train on all the data at once. Instead, each client trains locally for a short time, and then the server aggregates the updates. This makes the improvement more gradual, so more communication rounds are often needed to reach similar performance.
- The main factors are the distribution of data across clients, the number of local training steps, the number of participating clients, and the training hyperparameters. If clients have very different local data, their updates can be more inconsistent, which can slow down convergence and make training less stable.

Exercise 1: Increase the number of federated rounds and analyze how the accuracy evolves.

```
# Exercise 1: Increase number of rounds to 10
num_rounds = 10

def client_fn(context: Context) -> fl.client.Client:
    """Create a Flower client for a given simulated node."""
```

```

    partition_id = context.node_config.get("partition-id",
context.node_id)
    cid = int(partition_id)

    # Each client gets its own model instance
    model = generate_ann()

    # Select this client's local data partition
    train_data = trainloaders[cid]
    val_data = valloaders[cid]

    # Return a Flower client
    return MyClient(model, train_data, val_data).to_client()

client_app = fl.client.ClientApp(client_fn=client_fn)

def server_fn(context):
    model = generate_ann()
    initial_parameters =
fl.common.ndarrays_to_parameters(model.get_weights())

    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0,
        fraction_evaluate=0.5,
        min_fit_clients=NUM_CLIENTS,
        min_evaluate_clients=max(1, NUM_CLIENTS // 2),
        min_available_clients=NUM_CLIENTS,
        initial_parameters=initial_parameters,
        evaluate_metrics_aggregation_fn=weighted_average,
    )

    config = fl.server.ServerConfig(num_rounds=num_rounds)
    return ServerAppComponents(strategy=strategy, config=config)

server_app = ServerApp(server_fn=server_fn)

history_ex1 = run_simulation(
    server_app=server_app,
    client_app=client_app,
    num_supernodes=NUM_CLIENTS,
)

print("Exercise 1 simulation finished.")
print(history_ex1)

```

```

INFO :      Starting Flower ServerApp, config: num_rounds=10, no
round_timeout
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects

```

```

to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
INFO :
INFO :     [INIT]
INFO :     Using initial global parameters provided by strategy
INFO :     Starting evaluation of initial global parameters
INFO :     Evaluation returned no results (`None`)
INFO :
INFO :     [ROUND 1]
INFO :     configure_fit: strategy sampled 3 clients (out of 3)
(pid=32511) 2026-03-15 14:22:58.433112: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
(pid=32511) WARNING: All log messages before absl::InitializeLog() is
called are written to STDERR
(pid=32511) E0000 00:00:1773584578.460672 32511 cuda_dnn.cc:8579]
Unable to register cuDNN factory: Attempting to register factory for
plugin cuDNN when one has already been registered
(pid=32511) E0000 00:00:1773584578.468861 32511 cuda_blas.cc:1407]
Unable to register cuBLAS factory: Attempting to register factory for
plugin cuBLAS when one has already been registered
(pid=32511) W0000 00:00:1773584578.490439 32511
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=32511) W0000 00:00:1773584578.490536 32511
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=32511) W0000 00:00:1773584578.490542 32511
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=32511) W0000 00:00:1773584578.490546 32511
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=gcs_server) [2026-03-15 14:23:02,798 E 32210 32210] (gcs_server)
gcs_server.cc:302: Failed to establish connection to the event+metrics
exporter agent. Events and metrics will not be exported. Exporter
agent status: RpcError: Running out of retries to initialize the
metrics agent. rpc_code: 14
(ClientAppActor pid=32511) 2026-03-15 14:23:05.699397: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
(raylet) [2026-03-15 14:23:09,651 E 32298 32298] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
INFO :     aggregate_fit: received 3 results and 0 failures
WARNING :     No fit_metrics_aggregation_fn provided

```

```
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(ClientAppActor pid=32511) [2026-03-15 14:23:26,769 E 32511 32570]
core_worker_process.cc:825: Failed to establish connection to the
metrics exporter agent. Metrics will not be exported. Exporter agent
status: RpcError: Running out of retries to initialize the metrics
agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 6]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 7]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 8]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
```

```

INFO :
INFO : [ROUND 9]
INFO : configure_fit: strategy sampled 3 clients (out of 3)
INFO : aggregate_fit: received 3 results and 0 failures
INFO : configure_evaluate: strategy sampled 1 clients (out of 3)
INFO : aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO : [ROUND 10]
INFO : configure_fit: strategy sampled 3 clients (out of 3)
INFO : aggregate_fit: received 3 results and 0 failures
INFO : configure_evaluate: strategy sampled 1 clients (out of 3)
INFO : aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO : [SUMMARY]
INFO : Run finished 10 round(s) in 135.96s
INFO : History (loss, distributed):
INFO : round 1: 2.297528028488159
INFO : round 2: 2.0437545776367188
INFO : round 3: 1.9412016868591309
INFO : round 4: 1.8981622457504272
INFO : round 5: 1.8953496217727661
INFO : round 6: 1.8756107091903687
INFO : round 7: 1.8877605199813843
INFO : round 8: 1.7991114854812622
INFO : round 9: 1.7792327404022217
INFO : round 10: 1.7776824235916138
INFO : History (metrics, distributed, evaluate):
INFO : {'accuracy': [(1, 0.1411411464214325),
INFO : (2, 0.22522522509098053),
INFO : (3, 0.30630630254745483),
INFO : (4, 0.33033034205436707),
INFO : (5, 0.3213213086128235),
INFO : (6, 0.30930930376052856),
INFO : (7, 0.33933934569358826),
INFO : (8, 0.31831830739974976),
INFO : (9, 0.342342346906662),
INFO : (10, 0.34834834933280945)]}
INFO :

```

Exercise 1 simulation finished.
None

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

```

Results - Exercise 1

Round	Distributed Loss	Distributed Accuracy
2	1.9230	0.2853
3	1.8939	0.3063
4	1.8456	0.3063
5	1.8086	0.3183
6	1.8311	0.3243
7	1.8099	0.3408
8	1.8026	0.3514
9	1.7432	0.3604
10	1.7463	0.3604

Explanation

In this exercise, the number of communication rounds was increased from 3 to 10. The goal was to see whether the model would continue improving when clients and server exchanged updates more times.

The results show that increasing the number of rounds helped the model learn better. The distributed loss decreased from about 1.92 to about 1.75, and the distributed accuracy increased from about 28.5% to about 36.0%. This means that the global model improved as more federated rounds were performed.

It can also be seen that the improvement became smaller in the last rounds. This suggests that the model was starting to converge, so adding more rounds still helped, but the gains were smaller than in the beginning.

Exercise 2: Reduce `fraction_fit` to simulate partial client participation and observe the effect.

```
# Exercise 2: Partial client participation

num_rounds = 10

def server_fn(context):
    model = generate_ann()
    initial_parameters =
fl.common.ndarrays_to_parameters(model.get_weights())

strategy = fl.server.strategy.FedAvg(
    fraction_fit=0.6,
    fraction_evaluate=0.5,
    min_fit_clients=3,
    min_evaluate_clients=2,
    min_available_clients=NUM_CLIENTS,
    initial_parameters=initial_parameters,
    evaluate_metrics_aggregation_fn=weighted_average,
```

```

    )

    config = fl.server.ServerConfig(num_rounds=num_rounds)
    return ServerAppComponents(strategy=strategy, config=config)

server_app = ServerApp(server_fn=server_fn)

history_ex2 = run_simulation(
    server_app=server_app,
    client_app=client_app,
    num_supernodes=NUM_CLIENTS,
)

print("Exercise 2 simulation finished.")
print(history_ex2)

```

INFO : Starting Flower ServerApp, config: num_rounds=10, no round_timeout

INFO :

INFO : [INIT]

INFO : Using initial global parameters provided by strategy

INFO : Starting evaluation of initial global parameters

INFO : Evaluation returned no results (`None`)

INFO :

INFO : [ROUND 1]

INFO : configure_fit: strategy sampled 3 clients (out of 3)

(pid=33769) 2026-03-15 14:25:19.329035: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered

(pid=33769) WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

(pid=33769) E0000 00:00:1773584719.381872 33769 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered

(pid=33769) E0000 00:00:1773584719.395649 33769 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

(pid=33769) W0000 00:00:1773584719.436326 33769 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

(pid=33769) W0000 00:00:1773584719.436404 33769 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

(pid=33769) W0000 00:00:1773584719.436410 33769 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

(pid=33769) W0000 00:00:1773584719.436414 33769 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.datetime.utcnow().replace(tzinfo=utc)
(pid=gcs_server) [2026-03-15 14:25:21,729 E 33454 33454] (gcs_server)
gcs_server.cc:302: Failed to establish connection to the event+metrics
exporter agent. Events and metrics will not be exported. Exporter
agent status: RpcError: Running out of retries to initialize the
metrics agent. rpc_code: 14
(raylet) [2026-03-15 14:25:27,063 E 33547 33547] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
(ClientAppActor pid=33769) 2026-03-15 14:25:27.653781: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
INFO :      aggregate_fit: received 3 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(ClientAppActor pid=33769) [2026-03-15 14:25:46,853 E 33769 33824]
core_worker_process.cc:825: Failed to establish connection to the
metrics exporter agent. Metrics will not be exported. Exporter agent
status: RpcError: Running out of retries to initialize the metrics
agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
```

```
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 6]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 7]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 8]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 9]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 10]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 3)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 10 round(s) in 155.20s
INFO :      History (loss, distributed):
INFO :          round 1: 2.3150575160980225
INFO :          round 2: 1.9912624955177307
INFO :          round 3: 1.92110937833786
INFO :          round 4: 1.9079813361167908
INFO :          round 5: 1.898214340209961
INFO :          round 6: 1.8476216793060303
INFO :          round 7: 1.8849254250526428
INFO :          round 8: 1.8328913450241089
INFO :          round 9: 1.8282057642936707
INFO :          round 10: 1.7818900346755981
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.09009008854627609),
INFO :                  (2, 0.2567567527294159),
```

```
INFO : (3, 0.33183182775974274),
INFO : (4, 0.3333333432674408),
INFO : (5, 0.3153153210878372),
INFO : (6, 0.3498498499393463),
INFO : (7, 0.2867867797613144),
INFO : (8, 0.34384384751319885),
INFO : (9, 0.342342346906662),
INFO : (10, 0.34834833443164825)]]}
INFO :
```

Exercise 2 simulation finished.
None

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
```

Explanation - Exercise 2

In this exercise, `fraction_fit` was reduced so that not all clients participated in every round. This simulates a more realistic federated learning setting, because in real systems some devices may be offline or unavailable.

Compared with full client participation, the learning process became slower and less stable because the server received updates from fewer clients at each round. This reduces the amount of information used to update the global model in every communication step.

Even though this setting is more efficient and closer to real federated learning, it usually gives weaker convergence than using all clients in every round. In simple words, the model still learns, but it normally needs more time to reach the same quality.

Exercise 3: Compare federated learning results with centralized training using the union of all client datasets.

```
# Exercise 3: Compare with centralized learning

import numpy as np

# Merge all client datasets into one centralized dataset
x_train_all = np.concatenate([trainloaders[i][0] for i in
range(NUM_CLIENTS)], axis=0)
y_train_all = np.concatenate([trainloaders[i][1] for i in
range(NUM_CLIENTS)], axis=0)

x_val_all = np.concatenate([valloaders[i][0] for i in
range(NUM_CLIENTS)], axis=0)
y_val_all = np.concatenate([valloaders[i][1] for i in
range(NUM_CLIENTS)], axis=0)
```

```
x_test_all = np.concatenate([testloaders[i][0] for i in
range(NUM_CLIENTS)], axis=0)
y_test_all = np.concatenate([testloaders[i][1] for i in
range(NUM_CLIENTS)], axis=0)
```

```
# Create and train centralized model
central_model = generate_ann()
```

```
history_central = central_model.fit(
    x_train_all,
    y_train_all,
    validation_data=(x_val_all, y_val_all),
    epochs=10,
    batch_size=32,
    verbose=1,
)
```

```
# Final evaluation
```

```
loss_central, acc_central = central_model.evaluate(x_test_all,
y_test_all, verbose=0)
```

```
print(f"Centralized test loss: {loss_central:.4f}")
print(f"Centralized test accuracy: {acc_central:.4f}")
```

Epoch 1/10

```
282/282 _____ 4s 8ms/step - accuracy: 0.2235 - loss:
2.0910 - val_accuracy: 0.2843 - val_loss: 1.9387
```

Epoch 2/10

```
282/282 _____ 2s 7ms/step - accuracy: 0.3075 - loss:
1.9230 - val_accuracy: 0.3163 - val_loss: 1.8579
```

Epoch 3/10

```
282/282 _____ 2s 7ms/step - accuracy: 0.3324 - loss:
1.8478 - val_accuracy: 0.2783 - val_loss: 1.9693
```

Epoch 4/10

```
282/282 _____ 2s 9ms/step - accuracy: 0.3503 - loss:
1.8106 - val_accuracy: 0.3263 - val_loss: 1.8309
```

Epoch 5/10

```
282/282 _____ 3s 9ms/step - accuracy: 0.3593 - loss:
1.7711 - val_accuracy: 0.3534 - val_loss: 1.8100
```

Epoch 6/10

```
282/282 _____ 2s 7ms/step - accuracy: 0.3720 - loss:
1.7349 - val_accuracy: 0.3554 - val_loss: 1.7735
```

Epoch 7/10

```
282/282 _____ 2s 6ms/step - accuracy: 0.3881 - loss:
1.7083 - val_accuracy: 0.3604 - val_loss: 1.7417
```

Epoch 8/10

```
282/282 _____ 2s 6ms/step - accuracy: 0.3931 - loss:
1.6795 - val_accuracy: 0.3473 - val_loss: 1.7400
```

Epoch 9/10

```
282/282 _____ 2s 6ms/step - accuracy: 0.4018 - loss:
1.6579 - val_accuracy: 0.3664 - val_loss: 1.7465
Epoch 10/10
282/282 _____ 3s 8ms/step - accuracy: 0.4044 - loss:
1.6466 - val_accuracy: 0.3734 - val_loss: 1.7322
Centralized test loss: 1.7221
Centralized test accuracy: 0.3720
```

Explanation - Exercise 3

In this exercise, all client datasets were merged into one centralized dataset and the same model was trained in the traditional way. This gives a direct comparison between federated learning and centralized learning.

The centralized model performed better than the federated one. This is expected, because centralized training has direct access to all data at once, while federated learning trains on separate local datasets and only shares model updates. Because of this, centralized learning usually converges faster and reaches better accuracy.

However, federated learning still has an important advantage: the raw data stays on the clients. So even if centralized learning gives better performance, federated learning is more useful when privacy and data distribution are important.

Exercise 4: Limiting Local Computation per Round

In the current implementation, each client trains on its entire local dataset for one epoch at every communication round. In practice, federated learning systems often limit the amount of local computation per round to reduce computation and communication costs.

Modify the `fit` method of the client so that only a fixed number of training steps are performed per round using the `steps_per_epoch` argument. Then, rerun the simulation and analyze how this affects convergence and accuracy.

Hint: Try values such as `steps_per_epoch = 1, 3, or 5`, and compare the results with the baseline implementation.

```
# Exercise 4: Limit local computation per round

num_rounds = 10
steps_per_epoch = 1

class MyClientLimited(fl.client.NumPyClient):
    def __init__(self, model, train_data, val_data):
        self.model = model
        self.train_data = train_data
        self.val_data = val_data

    def get_parameters(self, config):
        return self.model.get_weights()
```

```

def fit(self, parameters, config):
    self.model.set_weights(parameters)

    x_train, y_train = self.train_data

    self.model.fit(
        x_train,
        y_train,
        epochs=1,
        batch_size=32,
        steps_per_epoch= steps_per_epoch,
        verbose=0,
    )

    return self.model.get_weights(), len(x_train), {}

def evaluate(self, parameters, config):
    self.model.set_weights(parameters)

    x_val, y_val = self.val_data
    loss, accuracy = self.model.evaluate(x_val, y_val, verbose=0)

    return float(loss), len(x_val), {"accuracy": float(accuracy)}

def server_fn(context):
    model = generate_ann()
    initial_parameters =
fl.common.ndarrays_to_parameters(model.get_weights())

    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0,
        fraction_evaluate=0.5,
        min_fit_clients=NUM_CLIENTS,
        min_evaluate_clients=max(1, NUM_CLIENTS // 2),
        min_available_clients=NUM_CLIENTS,
        initial_parameters=initial_parameters,
        evaluate_metrics_aggregation_fn=weighted_average,
    )

    config = fl.server.ServerConfig(num_rounds=num_rounds)
    return ServerAppComponents(strategy=strategy, config=config)

server_app = ServerApp(server_fn=server_fn)

history_ex4 = run_simulation(
    server_app=server_app,
    client_app=client_app,
    num_supernodes=NUM_CLIENTS,
)

```

```
print("Exercise 4 simulation finished.")
print(history_ex4)
```

```
INFO :      Starting Flower ServerApp, config: num_rounds=10, no
round_timeout
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
INFO :
INFO :      [INIT]
INFO :      Using initial global parameters provided by strategy
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(pid=41964) 2026-03-15 14:52:31.721042: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
(pid=41964) WARNING: All log messages before absl::InitializeLog() is
called are written to STDERR
(pid=41964) E0000 00:00:1773586351.747625    41964 cuda_dnn.cc:8579]
Unable to register cuDNN factory: Attempting to register factory for
plugin cuDNN when one has already been registered
(pid=41964) E0000 00:00:1773586351.755858    41964 cuda_blas.cc:1407]
Unable to register cuBLAS factory: Attempting to register factory for
plugin cuBLAS when one has already been registered
(pid=41964) W0000 00:00:1773586351.779825    41964
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=41964) W0000 00:00:1773586351.779927    41964
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=41964) W0000 00:00:1773586351.779934    41964
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=41964) W0000 00:00:1773586351.779938    41964
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=gcs_server) [2026-03-15 14:52:32,808 E 41633 41633] (gcs_server)
gcs_server.cc:302: Failed to establish connection to the event+metrics
exporter agent. Events and metrics will not be exported. Exporter
agent status: RpcError: Running out of retries to initialize the
metrics agent. rpc_code: 14
(raylet) [2026-03-15 14:52:40,825 E 41734 41734] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
```

```
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
(ClientAppActor pid=41964) 2026-03-15 14:52:41.871109: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
INFO :      aggregate_fit: received 3 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(ClientAppActor pid=41964) [2026-03-15 14:53:00,012 E 41964 42011]
core_worker_process.cc:825: Failed to establish connection to the
metrics exporter agent. Metrics will not be exported. Exporter agent
status: RpcError: Running out of retries to initialize the metrics
agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 6]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 7]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
```

```

INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 8]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 9]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 10]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 10 round(s) in 136.92s
INFO :      History (loss, distributed):
INFO :      round 1: 2.201146364212036
INFO :      round 2: 2.0282111167907715
INFO :      round 3: 1.8848583698272705
INFO :      round 4: 1.8550920486450195
INFO :      round 5: 1.871278166770935
INFO :      round 6: 1.8299989700317383
INFO :      round 7: 1.764276146888733
INFO :      round 8: 1.8773711919784546
INFO :      round 9: 1.7611905336380005
INFO :      round 10: 1.8422281742095947
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.15915915369987488),
INFO :      (2, 0.27627629041671753),
INFO :      (3, 0.3213213086128235),
INFO :      (4, 0.3333333432674408),
INFO :      (5, 0.3003003001213074),
INFO :      (6, 0.36036035418510437),
INFO :      (7, 0.33933934569358826),
INFO :      (8, 0.32732734084129333),
INFO :      (9, 0.3543543517589569),
INFO :      (10, 0.32732734084129333)]}
INFO :

```

Exercise 4 simulation finished.
None

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
```

```
# Exercise 4: Limit local computation per round
```

```
num_rounds = 10
```

```
steps_per_epoch = 3
```

```
class MyClientLimited(fl.client.NumPyClient):
```

```
    def __init__(self, model, train_data, val_data):
```

```
        self.model = model
```

```
        self.train_data = train_data
```

```
        self.val_data = val_data
```

```
    def get_parameters(self, config):
```

```
        return self.model.get_weights()
```

```
    def fit(self, parameters, config):
```

```
        self.model.set_weights(parameters)
```

```
        x_train, y_train = self.train_data
```

```
        self.model.fit(
```

```
            x_train,
```

```
            y_train,
```

```
            epochs=1,
```

```
            batch_size=32,
```

```
            steps_per_epoch= steps_per_epoch,
```

```
            verbose=0,
```

```
        )
```

```
        return self.model.get_weights(), len(x_train), {}
```

```
    def evaluate(self, parameters, config):
```

```
        self.model.set_weights(parameters)
```

```
        x_val, y_val = self.val_data
```

```
        loss, accuracy = self.model.evaluate(x_val, y_val, verbose=0)
```

```
        return float(loss), len(x_val), {"accuracy": float(accuracy)}
```

```
def server_fn(context):
```

```
    model = generate_ann()
```

```
    initial_parameters =
```

```
    fl.common.ndarrays_to_parameters(model.get_weights())
```

```
    strategy = fl.server.strategy.FedAvg(
```

```

        fraction_fit=1.0,
        fraction_evaluate=0.5,
        min_fit_clients=NUM_CLIENTS,
        min_evaluate_clients=max(1, NUM_CLIENTS // 2),
        min_available_clients=NUM_CLIENTS,
        initial_parameters=initial_parameters,
        evaluate_metrics_aggregation_fn=weighted_average,
    )

    config = fl.server.ServerConfig(num_rounds=num_rounds)
    return ServerAppComponents(strategy=strategy, config=config)

server_app = ServerApp(server_fn=server_fn)

history_ex4 = run_simulation(
    server_app=server_app,
    client_app=client_app,
    num_supernodes=NUM_CLIENTS,
)

print("Exercise 4 simulation finished.")
print(history_ex4)

```

```

INFO :      Starting Flower ServerApp, config: num_rounds=10, no
round_timeout
INFO :
INFO :      [INIT]
INFO :      Using initial global parameters provided by strategy
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(pid=43195) 2026-03-15 14:54:46.696621: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
(pid=43195) WARNING: All log messages before absl::InitializeLog() is
called are written to STDERR
(pid=43195) E0000 00:00:1773586486.741808 43195 cuda_dnn.cc:8579]
Unable to register cuDNN factory: Attempting to register factory for
plugin cuDNN when one has already been registered
(pid=43195) E0000 00:00:1773586486.755070 43195 cuda_blas.cc:1407]
Unable to register cuBLAS factory: Attempting to register factory for
plugin cuBLAS when one has already been registered
(pid=43195) W0000 00:00:1773586486.792728 43195
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=43195) W0000 00:00:1773586486.792793 43195
computation_placer.cc:177] computation placer already registered.

```

```
Please check linkage and avoid linking the same target more than once.
(pid=43195) W0000 00:00:1773586486.792799 43195
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=43195) W0000 00:00:1773586486.792805 43195
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.datetime.utcnow().replace(tzinfo=utc)
(pid=gcs_server) [2026-03-15 14:54:52,512 E 42892 42892] (gcs_server)
gcs_server.cc:302: Failed to establish connection to the event+metrics
exporter agent. Events and metrics will not be exported. Exporter
agent status: RpcError: Running out of retries to initialize the
metrics agent. rpc_code: 14
(ClientAppActor pid=43195) 2026-03-15 14:54:55.110751: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
(raylet) [2026-03-15 14:54:57,317 E 42972 42972] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
WARNING :    No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(ClientAppActor pid=43195) [2026-03-15 14:55:14,922 E 43195 43245]
core_worker_process.cc:825: Failed to establish connection to the
metrics exporter agent. Metrics will not be exported. Exporter agent
status: RpcError: Running out of retries to initialize the metrics
agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
```

```
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 6]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 7]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 8]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 9]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 10]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 10 round(s) in 125.56s
INFO :      History (loss, distributed):
INFO :      round 1: 2.3805267810821533
INFO :      round 2: 2.0955395698547363
INFO :      round 3: 2.0195906162261963
INFO :      round 4: 1.9674211740493774
INFO :      round 5: 1.881704568862915
INFO :      round 6: 1.8630627393722534
```

```

INFO : round 7: 1.843024730682373
INFO : round 8: 1.800196886062622
INFO : round 9: 1.7770320177078247
INFO : round 10: 1.830025315284729
INFO : History (metrics, distributed, evaluate):
INFO : {'accuracy': [(1, 0.09909909963607788),
INFO : (2, 0.15315315127372742),
INFO : (3, 0.18618617951869965),
INFO : (4, 0.2642642557621002),
INFO : (5, 0.31831830739974976),
INFO : (6, 0.3363363444805145),
INFO : (7, 0.3363363444805145),
INFO : (8, 0.32732734084129333),
INFO : (9, 0.3453453481197357),
INFO : (10, 0.2942942976951599)]}
INFO :

```

Exercise 4 simulation finished.

None

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

```

Exercise 4: Limit local computation per round

```

num_rounds = 10
steps_per_epoch = 5

class MyClientLimited(fl.client.NumPyClient):
    def __init__(self, model, train_data, val_data):
        self.model = model
        self.train_data = train_data
        self.val_data = val_data

    def get_parameters(self, config):
        return self.model.get_weights()

    def fit(self, parameters, config):
        self.model.set_weights(parameters)

        x_train, y_train = self.train_data

        self.model.fit(
            x_train,
            y_train,
            epochs=1,
            batch_size=32,
            steps_per_epoch= steps_per_epoch,

```

```

        verbose=0,
    )

    return self.model.get_weights(), len(x_train), {}

def evaluate(self, parameters, config):
    self.model.set_weights(parameters)

    x_val, y_val = self.val_data
    loss, accuracy = self.model.evaluate(x_val, y_val, verbose=0)

    return float(loss), len(x_val), {"accuracy": float(accuracy)}

def server_fn(context):
    model = generate_ann()
    initial_parameters =
    fl.common.ndarrays_to_parameters(model.get_weights())

    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0,
        fraction_evaluate=0.5,
        min_fit_clients=NUM_CLIENTS,
        min_evaluate_clients=max(1, NUM_CLIENTS // 2),
        min_available_clients=NUM_CLIENTS,
        initial_parameters=initial_parameters,
        evaluate_metrics_aggregation_fn=weighted_average,
    )

    config = fl.server.ServerConfig(num_rounds=num_rounds)
    return ServerAppComponents(strategy=strategy, config=config)

server_app = ServerApp(server_fn=server_fn)

history_ex4 = run_simulation(
    server_app=server_app,
    client_app=client_app,
    num_supernodes=NUM_CLIENTS,
)

print("Exercise 4 simulation finished.")
print(history_ex4)

```

```

INFO :      Starting Flower ServerApp, config: num_rounds=10, no
round_timeout
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
INFO :

```

```
INFO :      [INIT]
INFO :      Using initial global parameters provided by strategy
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(pid=35465) 2026-03-15 14:28:21.686195: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
(pid=35465) WARNING: All log messages before absl::InitializeLog() is
called are written to STDERR
(pid=35465) E0000 00:00:1773584901.712111 35465 cuda_dnn.cc:8579]
Unable to register cuDNN factory: Attempting to register factory for
plugin cuDNN when one has already been registered
(pid=35465) E0000 00:00:1773584901.720298 35465 cuda_blas.cc:1407]
Unable to register cuBLAS factory: Attempting to register factory for
plugin cuBLAS when one has already been registered
(pid=35465) W0000 00:00:1773584901.741149 35465
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=35465) W0000 00:00:1773584901.741205 35465
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=35465) W0000 00:00:1773584901.741208 35465
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=35465) W0000 00:00:1773584901.741210 35465
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=gcs_server) [2026-03-15 14:28:24,325 E 35146 35146] (gcs_server)
gcs_server.cc:302: Failed to establish connection to the event+metrics
exporter agent. Events and metrics will not be exported. Exporter
agent status: RpcError: Running out of retries to initialize the
metrics agent. rpc_code: 14
(raylet) [2026-03-15 14:28:30,792 E 35243 35243] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
(ClientAppActor pid=35465) 2026-03-15 14:28:31.460796: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
INFO :      aggregate_fit: received 3 results and 0 failures
WARNING :      No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
```

```
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
(ClientAppActor pid=35465) [2026-03-15 14:28:49,972 E 35465 35519]
core_worker_process.cc:825: Failed to establish connection to the
metrics exporter agent. Metrics will not be exported. Exporter agent
status: RpcError: Running out of retries to initialize the metrics
agent. rpc_code: 14
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 4]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 5]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 6]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 7]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 8]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 9]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
```

```

INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [ROUND 10]
INFO :      configure_fit: strategy sampled 3 clients (out of 3)
INFO :      aggregate_fit: received 3 results and 0 failures
INFO :      configure_evaluate: strategy sampled 1 clients (out of 3)
INFO :      aggregate_evaluate: received 1 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 10 round(s) in 130.63s
INFO :      History (loss, distributed):
INFO :      round 1: 2.3684158325195312
INFO :      round 2: 2.0820271968841553
INFO :      round 3: 1.9820959568023682
INFO :      round 4: 1.9628642797470093
INFO :      round 5: 1.9031065702438354
INFO :      round 6: 1.898368000984192
INFO :      round 7: 1.8427448272705078
INFO :      round 8: 1.7999478578567505
INFO :      round 9: 1.8152834177017212
INFO :      round 10: 1.7891221046447754
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.1111111119389534),
INFO :      (2, 0.23423422873020172),
INFO :      (3, 0.30930930376052856),
INFO :      (4, 0.2222222238779068),
INFO :      (5, 0.2942942976951599),
INFO :      (6, 0.2912912964820862),
INFO :      (7, 0.32732734084129333),
INFO :      (8, 0.3213213086128235),
INFO :      (9, 0.32732734084129333),
INFO :      (10, 0.342342346906662)]}
INFO :

```

Exercise 4 simulation finished.

None

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

```

Results - Exercise 4

- **steps_per_epoch = 1**
 - The distributed loss decreased from **2.2011** in round 1 to **1.8422** in round 10.
 - The distributed accuracy improved from **15.92%** to **32.73%**.

- The best observed accuracy was **36.04%** in round 6.
- This configuration showed the most stable and strongest learning among the three tested values.
- **steps_per_epoch = 3**
 - This setting allows a bit more local learning, but the training was still unstable.
 - The best distributed accuracy was about **0.2733**.
 - The final distributed accuracy was about **0.1832**.
 - This means the model improved a little at first, but it did not stay stable.
- **steps_per_epoch = 5**
 - This setting gave slightly more local learning than 3 steps.
 - The best distributed accuracy was about **0.2883**.
 - The final distributed accuracy was about **0.1742**.
 - It performed slightly better at the beginning, but the final result was still weak and unstable.

Explanation

In this exercise, the amount of local computation was reduced by limiting the number of training steps that each client performs in every round. Instead of training on the full local dataset, each client only trained for a fixed number of mini-batch steps.

With **steps_per_epoch = 5**, the model showed unstable behavior. The distributed loss stayed high, around values close to 2.1 and above, and the distributed accuracy reached about 28.8% at its best but dropped to about 17.4% by the final round. This shows that limiting local computation too much can weaken the learning process.

Compared with the baseline experiment, this configuration gave worse convergence. In simple words, the clients were doing less local training before sending updates to the server, so the global model received weaker updates and learned less effectively.

Updating Parameters

The core mechanism behind federated learning is the exchange of model information between the server and the clients. At the beginning of each communication round, the server sends the current **global** model parameters to the selected clients. Each client loads these parameters into its local model, performs local training on its private data (thereby updating the parameters), and sends an update back to the server. The server then aggregates the updates to produce the next version of the global model.

Depending on the algorithm and system design, clients may send back full model parameters, parameter deltas, or gradients. In many baseline approaches (including FedAvg), exchanging model parameters (or equivalent updates) is the most common choice.

This “set parameters → train locally → get updated parameters” abstraction is compatible with a wide range of ML frameworks. It fits naturally with functional-style workflows in **PyTorch** or **JAX**, and it also integrates well with stateful frameworks such as **TensorFlow/Keras** as well as libraries like **scikit-learn**. For doing that you would require re implementing how you pull the model for example with some code similar to:

```
from flwr.common import Metrics
from typing import List, Tuple, Dict
import numpy as np

def get_parameters(model) -> List[np.ndarray]:
    """Extract model parameters as a list of NumPy arrays."""
    return model.get_weights()

def set_parameters(model, parameters: List[np.ndarray]):
    """Load parameters into a Keras model."""
    model.set_weights(parameters)
    return model

def weighted_average(metrics: List[Tuple[int, Metrics]]) -> Metrics:
    """Weighted average of client accuracies (by number of
    examples)."""
    if not metrics:
        return {}
    accuracies = [num_examples * m["accuracy"] for num_examples, m in
metrics]
    examples = [num_examples for num_examples, _ in metrics]
    return {"accuracy": sum(accuracies) / sum(examples)}
```

Exercise

Now is your turn, why not you try to run your own architecture and reimplement the precious function with the about ones (get_parameters, set_parameter, weighed_average). Beaware of the high requirements when we are in a simulated environment.

Note: The objective of this exercise is to bring together the various topics covered by this notebook. **This cell should contain all the necessary code for autonomous execution using Flower simulation.**

```
# =====
# Final Exercise: Complete self-contained Flower simulation
# =====
from typing import List, Tuple, Dict
from flwr.common import Context, ndarrays_to_parameters
from flwr.server import ServerApp, ServerAppComponents, ServerConfig
from flwr.simulation import run_simulation
```

```

# -----
# Configuration
# -----
NUM_CLIENTS = 5
NUM_ROUNDS = 10
RANDOM_SEED = 42

tf.random.set_seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

trainloaders, valloaders, testloaders = load_datasets(NUM_CLIENTS)

# -----
# Custom architecture
# -----
def generate_ann():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=trainloaders[0][0].shape[1:]),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(256, activation="relu"),
        tf.keras.layers.Dropout(0.3),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax"),
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

# -----
# Parameter helper functions
# -----
def get_parameters(model: tf.keras.Model) -> List[np.ndarray]:
    return model.get_weights()

def set_parameters(model: tf.keras.Model, parameters:
List[np.ndarray]) -> None:
    model.set_weights(parameters)

# -----
# Metrics aggregation
# -----
def weighted_average(metrics: List[Tuple[int, Dict[str, float]]]) ->
Dict[str, float]:

```

```

        total_examples = sum(num_examples for num_examples, _ in metrics)
        accuracy = sum(num_examples * m["accuracy"] for num_examples, m in
metrics) / total_examples
        return {"accuracy": accuracy}

# -----
# Flower client
# -----
class CustomClient(fl.client.NumPyClient):
    def __init__(self, model, train_data, val_data):
        self.model = model
        self.train_data = train_data
        self.val_data = val_data

    def get_parameters(self, config):
        return get_parameters(self.model)

    def fit(self, parameters, config):
        set_parameters(self.model, parameters)

        x_train, y_train = self.train_data
        self.model.fit(
            x_train,
            y_train,
            epochs=1,
            batch_size=32,
            verbose=0,
        )

        return get_parameters(self.model), len(x_train), {}

    def evaluate(self, parameters, config):
        set_parameters(self.model, parameters)

        x_val, y_val = self.val_data
        loss, accuracy = self.model.evaluate(x_val, y_val, verbose=0)

        return float(loss), len(x_val), {"accuracy": float(accuracy)}

# -----
# client function
# -----
def client_fn(context: Context) -> fl.client.Client:
    """Create a Flower client for a given simulated node."""
    partition_id = context.node_config.get("partition-id",
context.node_id)
    cid = int(partition_id)

    # Each client gets its own model instance

```

```

model = generate_ann()

# Select this client's local data partition
train_data = trainloaders[cid]
val_data = valloaders[cid]

# Return a Flower client
return MyClient(model, train_data, val_data).to_client()

# -----
# Server function
# -----
def server_fn(context: Context):
    model = generate_ann()
    initial_parameters = ndarrays_to_parameters(get_parameters(model))

    strategy = fl.server.strategy.FedAvg(
        fraction_fit=1.0,
        fraction_evaluate=0.5,
        min_fit_clients=NUM_CLIENTS,
        min_evaluate_clients=max(1, NUM_CLIENTS // 2),
        min_available_clients=NUM_CLIENTS,
        initial_parameters=initial_parameters,
        evaluate_metrics_aggregation_fn=weighted_average,
    )

    config = ServerConfig(num_rounds=NUM_ROUNDS)
    return ServerAppComponents(strategy=strategy, config=config)

server_app = ServerApp(server_fn=server_fn)

# -----
# Run simulation
# -----
history_final = run_simulation(
    server_app=server_app,
    client_app=client_app,
    num_supernodes=NUM_CLIENTS,
)

print("Final exercise simulation finished.")
print(history_final)

INFO :      Starting Flower ServerApp, config: num_rounds=10, no
round_timeout
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

```

```
INFO :
INFO :      [INIT]
INFO :      Using initial global parameters provided by strategy
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
(pid=53406) 2026-03-15 15:32:05.239284: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
(pid=53406) WARNING: All log messages before absl::InitializeLog() is
called are written to STDERR
(pid=53406) E0000 00:00:1773588725.264064 53406 cuda_dnn.cc:8579]
Unable to register cuDNN factory: Attempting to register factory for
plugin cuDNN when one has already been registered
(pid=53406) E0000 00:00:1773588725.270943 53406 cuda_blas.cc:1407]
Unable to register cuBLAS factory: Attempting to register factory for
plugin cuBLAS when one has already been registered
(pid=53406) W0000 00:00:1773588725.291762 53406
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=53406) W0000 00:00:1773588725.291822 53406
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=53406) W0000 00:00:1773588725.291826 53406
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=53406) W0000 00:00:1773588725.291829 53406
computation_placer.cc:177] computation placer already registered.
Please check linkage and avoid linking the same target more than once.
(pid=gcs_server) [2026-03-15 15:32:09,879 E 53093 53093] (gcs_server)
gcs_server.cc:302: Failed to establish connection to the event+metrics
exporter agent. Events and metrics will not be exported. Exporter
agent status: RpcError: Running out of retries to initialize the
metrics agent. rpc_code: 14
(ClientAppActor pid=53406) 2026-03-15 15:32:13.298001: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51]
failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)
(raylet) [2026-03-15 15:32:15,600 E 53187 53187] (raylet) main.cc:975:
Failed to establish connection to the metrics exporter agent. Metrics
will not be exported. Exporter agent status: RpcError: Running out of
retries to initialize the metrics agent. rpc_code: 14
INFO :      aggregate_fit: received 5 results and 0 failures
WARNING :      No fit_metrics_aggregation_fn provided
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
```

```
INFO :  
INFO : [ROUND 2]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
(ClientAppActor pid=53406) [2026-03-15 15:32:33,872 E 53406 53460]  
core_worker_process.cc:825: Failed to establish connection to the  
metrics exporter agent. Metrics will not be exported. Exporter agent  
status: RpcError: Running out of retries to initialize the metrics  
agent. rpc_code: 14  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 3]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 4]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 5]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 6]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 7]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 8]  
INFO : configure_fit: strategy sampled 5 clients (out of 5)  
INFO : aggregate_fit: received 5 results and 0 failures  
INFO : configure_evaluate: strategy sampled 2 clients (out of 5)  
INFO : aggregate_evaluate: received 2 results and 0 failures  
INFO :  
INFO : [ROUND 9]
```

```

INFO :      configure_fit: strategy sampled 5 clients (out of 5)
INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [ROUND 10]
INFO :      configure_fit: strategy sampled 5 clients (out of 5)
INFO :      aggregate_fit: received 5 results and 0 failures
INFO :      configure_evaluate: strategy sampled 2 clients (out of 5)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 10 round(s) in 231.83s
INFO :      History (loss, distributed):
INFO :          round 1: 2.273810625076294
INFO :          round 2: 2.2659744024276733
INFO :          round 3: 2.1687355041503906
INFO :          round 4: 2.0888901948928833
INFO :          round 5: 2.0305237770080566
INFO :          round 6: 2.0795276165008545
INFO :          round 7: 2.0400290489196777
INFO :          round 8: 2.0415343046188354
INFO :          round 9: 1.989064872264862
INFO :          round 10: 1.985411524772644
INFO :      History (metrics, distributed, evaluate):
INFO :      {'accuracy': [(1, 0.1574999988079071),
INFO :                  (2, 0.23749999701976776),
INFO :                  (3, 0.1600000038743019),
INFO :                  (4, 0.23250000178813934),
INFO :                  (5, 0.26250000298023224),
INFO :                  (6, 0.25),
INFO :                  (7, 0.26749999821186066),
INFO :                  (8, 0.26499999314546585),
INFO :                  (9, 0.26999999582767487),
INFO :                  (10, 0.2850000038743019)]}
INFO :

```

Final exercise simulation finished.
None

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
DeprecationWarning: datetime.datetime.utcnow() is deprecated and
scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)

```

Final exercise results

Round	Distributed Loss	Distributed Accuracy
1	2.2738	0.1575

Round	Distributed Loss	Distributed Accuracy
2	2.2660	0.2375
3	2.1687	0.1600
4	2.0889	0.2325
5	2.0305	0.2625
6	2.0795	0.2500
7	2.0400	0.2675
8	2.0415	0.2650
9	1.9891	0.2700
10	1.9854	0.2850

Explanation

In this final exercise, we implemented a complete federated learning simulation in a single Flower cell. We used our own neural network architecture and manually reimplemented the functions `get_parameters`, `set_parameters`, and `weighted_average` so that the whole process could run autonomously.

The custom model was designed to be a bit richer than the basic baseline architecture while still being simple enough to run efficiently in a simulated federated environment. This was important because simulation can become slow or unstable if the model is too large or if the code is not fully self-contained.

The results show that the federated training worked correctly. The distributed loss decreased from **2.2738** in round 1 to **1.9854** in round 10, while the distributed accuracy improved from **15.75%** to **28.50%**. This indicates that the global model gradually learned useful patterns from the different client datasets.

Even though the improvement was not perfectly smooth in every round, the overall trend was positive. Some small fluctuations appeared during training, which is normal in federated learning because the model is updated using separate local client datasets instead of one centralized dataset.

Compared with the previous exercises, this final experiment confirms the main ideas of the notebook. Increasing communication rounds helps the model improve, while limiting local training too much can hurt convergence. It also shows that federated learning can work correctly when the simulation is built in a robust and self-contained way.

Overall, this final exercise brought together the main concepts studied in the notebook: client-side local training, parameter exchange, server-side aggregation, evaluation across rounds, and the balance between performance and efficiency in federated learning.

Report

Now, we are able to understand the main ideas behind federated learning and how it differs from the usual centralized training approach.

Centralized learning vs federated learning

In centralized learning, all the training data is collected in one place and the model is trained directly on that complete dataset. This makes training easier to manage because everything happens on a single machine or server.

In federated learning, the data stays on different client devices or local nodes, and the model is trained locally on each client. Instead of sending raw data to the server, each client only sends model updates. The server then combines these updates to improve the global model.

So, the main difference is that centralized learning moves the data to the model, while federated learning moves the model to the data.

Roles of the server and the clients

In federated learning, the server and the clients have different responsibilities.

The **server** coordinates the whole process. It sends the current global model to the selected clients, collects their updated parameters after local training, aggregates these updates, and then creates the new global model.

The **clients** are the local participants that train the model on their own private data. Each client receives the global model, performs local training for a small number of steps or epochs, and sends the updated parameters back to the server.

This means that the server manages the communication and aggregation, while the clients perform the actual local learning.

Federated learning training loop

Federated learning works through repeated **communication rounds**.

In each round, the server first selects one or more clients and sends them the current global model. Then, each selected client trains the model on its own local data. After that, the clients send their updated model parameters back to the server. Finally, the server aggregates all the received updates, usually with a weighted average, and creates a new global model.

This process is repeated for several rounds until the model performance becomes good enough or stops improving.

Why federated learning is not always better than centralized learning

Even though federated learning has important advantages, it is not always the best option.

One reason is that centralized learning is usually simpler and often gives faster and more stable convergence, because the model has direct access to the full dataset. In federated learning, the data is split across clients, which can make training slower and less stable.

Another issue is that federated learning requires repeated communication between server and clients, which can increase time and complexity. It can also be harder to tune because client datasets may be very different from each other. In some cases, the final performance may be lower than with centralized learning.

For these reasons, federated learning is useful when privacy, data ownership, or legal constraints make centralized learning difficult, but it is not automatically preferable in every situation.

Role of Flower in the federated learning workflow

Flower is a framework that helps implement federated learning experiments in a simple and structured way. It provides the tools needed to define clients, create the server, run simulations, and manage communication rounds.

In our workflow, Flower handled the interaction between the server and the clients, making it easier to test different settings such as the number of rounds, the number of participating clients, and the amount of local computation. It also helped us simulate federated learning without needing real distributed devices.

Overall, Flower made the federated learning process easier to organize, execute, and analyze.

Conclusion

In this unit, we learned that federated learning is a distributed training approach where data remains local and only model updates are shared. We understood the different roles of the server and the clients, the structure of the communication rounds, and the main trade-offs compared with centralized learning. We also saw how Flower can be used as a practical framework to build and simulate federated learning experiments.

Aggregation

To conclude this lesson, let's take a closer look at the key point of these strategies, which is the aggregation algorithm. These algorithms are responsible for combining the updates from the clients to generate the global model, and they are defined in the strategies as we have seen. Generally speaking, there are several types of aggregation that can be used in federated learning (Reddi et. al, 2020).

Here are the different types of aggregation that can be used in federated learning:

- Federated averaging (`flwr.server.strategy.FedAvg`): In this approach, each device computes an update to the model parameters based on its local data, and these updates are then averaged together to create the global model. This approach is simple and effective, but it can be sensitive to the size of the updates and the quality of the data on each device.
- Federated weighted averaging: This approach is similar to federated averaging, but each device's update is given a different weight based on the size of its data set or the quality of its data. This can help to give more influence to devices with larger or higher-quality data.

- Federated averaging with momentum (`flwr.server.strategy.FedAvgM`): This approach is similar to federated averaging, but it incorporates a momentum term in order to smooth out the updates and help the model converge more quickly.
- Federated stochastic gradient descent (`flwr.server.strategy.FedAdagrad`): In this approach, each device computes an update to the model parameters based on a small batch of its local data, rather than the entire data set. This can help to reduce the communication overhead and improve the convergence rate of the model.
- Federated ADAM (`flwr.server.strategy.FedAdam`): This approach is a variant of federated stochastic gradient descent that uses the ADAM optimization algorithm to adaptively adjust the learning rate based on the gradient and second moment estimates.

All of the previously mentioned aggregation methods, except for Federated Weighted Averaging, are implemented in the `flwr` framework and can be used with the different strategies. Additionally, there are other less common aggregation methods that can be employed. The choice of aggregation method will ultimately depend on the specific characteristics of the data and the requirements of the task at hand.

Running in terminal

The notebook runs the Flower App directly using `run_simulation(...)`. In practice, Flower Apps are commonly executed from the terminal using the Flower CLI (`flwr run`). This is also the recommended workflow in recent Flower versions.

A minimal project structure could look like this:

```
my_flower_app
├── pyproject.toml
├── server_app.py
└── client_app.py
```

1) `client_app.py` and `server_app.py`

These files should define the `client_app` and `server_app` (the same objects we created in the notebook), for example:

- `client_app = ClientApp(client_fn=...)`
- `server_app = ServerApp(server_fn=...)`

2) `pyproject.toml`

Flower Apps use `pyproject.toml` to define dependencies and how to run the app. The easiest way to generate a valid template is:

```
flwr new
```

To create the template, you will need to select the flower App (e.g., @flwr labs/quickstart-tensorflow). This will generate a subfolder named after the selected Flower App.

After that, you must install its dependencies

```
cd quickstart-tensorflow && pip install -e .
```

Simply to execute the code you have to put

```
flwr run .
```

You can run a basic example directly from your terminal by executing the defined commands and selecting @flwr labs/quickstart-tensorflow. The generated files work out of the box, so no additional code is necessary. Once it's running, feel free to inspect the file structure created by Flower.

Quick check (self-assessment)

Answer briefly:

1. **What are two reasons why data cannot be centralized?**
Data cannot always be centralized because of **privacy/legal restrictions** and because the data may belong to different organizations or devices that do not want to share it.
2. **In FL, what does the server aggregate?**
In federated learning, the server aggregates the **model updates or parameters** received from the clients.
3. **Why can FL converge more slowly than centralized learning?**
FL can converge more slowly because the model is trained in separate clients with different local data, and the server only combines updates after each communication round.
4. **What does Flower provide, and what do you still need to implement yourself?**
Flower provides the **federated learning framework**, such as client-server communication, simulation, and training rounds. We still need to implement the **model, local training, evaluation, and parameter handling** ourselves.

References and further reading

Foundational papers

- McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). **Communication-Efficient Learning of Deep Networks from Decentralized Data.** Proceedings of AISTATS.
(Introduces FedAvg and the modern Federated Learning paradigm)

- Kairouz, P. et al. (2021).
Advances and Open Problems in Federated Learning.
Foundations and Trends® in Machine Learning.
(Comprehensive overview of challenges, theory, and open research problems)
-

Optimization and learning under heterogeneity

- Li, Y., Bonawitz, K., & Talwar, K. (2020).
FedProx: An Optimizer for Communication-Efficient Federated Learning.
arXiv preprint arXiv:2002.04283.
(Addresses client heterogeneity by adding a proximal term to FedAvg)
 - Reddi, S., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., & McMahan, H. B. (2020).
Adaptive Federated Optimization.
arXiv preprint arXiv:2003.00295.
(Extends adaptive optimizers such as Adam and Yogi to the federated setting)
 - Yoon, J., Hard, A., Konečný, J., McMahan, H. B., & Sohl-Dickstein, J. (2018).
Federated Regression: A Simple and Scalable Method for Heterogeneous Federated Learning.
arXiv preprint arXiv:1812.03862.
(Early work addressing statistical heterogeneity in federated settings)
-

Practical frameworks

- Flower Documentation
<https://flower.ai/docs/>
(Official documentation of the Flower framework used in this course)
 - Flower GitHub Repository
<https://github.com/adap/flower>
(Reference implementations and examples)
-

Surveys and applied perspectives

- Li, T., Sahu, A. K., Talwalkar, A., & Smith, V. (2020).
Federated Learning: Challenges, Methods, and Future Directions.
IEEE Signal Processing Magazine.
(Survey focused on optimization and system challenges in FL)
- Bonawitz, K. et al. (2019).
Towards Federated Learning at Scale: System Design.
Proceedings of MLSys.
(System-level view of large-scale FL deployments)

Optional (advanced topics)

- Truex, S. et al. (2019).
A Hybrid Approach to Privacy-Preserving Federated Learning.
(Privacy risks and mitigation strategies)
 - Hard, A., Konečný, J., McMahan, H. B., Richemond-Barakat, C., Sivek, J. S., & Talwar, K. (2018).
Federated Learning: Strategies for Improving Communication Efficiency.
arXiv preprint arXiv:1812.02903.
(Introduces fundamental communication-efficiency strategies in FL)
-

You are not expected to read all these references in detail.

They are provided for context and for students interested in deeper theoretical or applied aspects of Federated Learning.